



中山大學
SUN YAT-SEN UNIVERSITY

实验七：MPI 并行快速傅里 叶变换

并行程序设计与算法实验报告

2026-05-18

元朗曦 23336294

计算机学院 计算机科学与技术

计八

一 实验目的

本实验以串行快速傅里叶变换程序 `fft_serial.cpp` 为基础，使用 MPI 将其改造为分布式内存环境下的并程序，并进一步比较不同通信组织方式对程序性能的影响。通过本实验，希望达到以下目的：

1. 理解快速傅里叶变换中蝶形运算的分阶段结构，以及其适合并行化的原因。
2. 掌握 MPI 中进程划分、点对点通信、同步计时与全局归约等基本用法。
3. 使用 `MPI_Pack` 与 `MPI_Unpack` 将离散通信数据重组为连续缓冲区，并分析打包通信对程序性能的影响。
4. 在改变问题规模与进程数的条件下观察并行效率，同时了解使用 Valgrind Massif 分析并行程序内存消耗的方法。

二 实验原理

2.1 快速傅里叶变换

对于长度为 N 的离散序列 x_n ，离散傅里叶变换定义为：

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi k \frac{n}{N}}, \quad k = 0, 1, \dots, N-1$$

直接计算需要 $O(N^2)$ 次复数运算。若 N 为 2 的幂，Cooley-Tukey FFT 可将问题递归拆分为偶数项与奇数项两部分，再通过蝶形运算合并结果，使复杂度降低为 $O(N \log N)$ 。一次蝶形运算可写为：

$$y_0 = a + \omega b, \quad y_1 = a - \omega b$$

其中 ω 为旋转因子。对长度为 N 的序列，FFT 共有 $\log_2 N$ 个阶段，每个阶段包含 $\frac{N}{2}$ 组相互独立的蝶形，因此天然适合并行执行。

2.2 分布式 FFT 的数据划分

实验将长度为 N 的复数序列按连续区间均分给 P 个 MPI 进程，每个进程持有 $\frac{N}{P}$ 个复数。程序要求 N 与 P 都为 2 的幂，且 P 能整除 N 。当蝶形跨度小于本地数据块大小时，运算完全在本地完成；当跨度超过本地块大小时，当前进程需要与伙伴进程交换数据。

若某一阶段的半跨度为 h ，本地块长度为 $L = \frac{N}{P}$ ，则跨进程阶段中的伙伴进程可表示为：

```
partner = rank xor (h / L)
```

这种按二进制位翻转寻找伙伴的方式，与 FFT 各阶段不断扩大合并范围的结构一致。随着阶段推进，通信从进程内部逐渐扩展到更远的伙伴进程。

2.3 DIF 与 DIT 的配合

若前向与反向变换都使用普通 DIT 形式，则通常需要额外处理位逆序排列。为减少一次全局重排，实验程序采用前向 DIF 与反向 DIT 的组合：

1. 前向 DIF 从自然序输入出发，输出位逆序频域结果；
2. 反向 DIT 正好接收位逆序输入，并恢复自然序时域结果。

这样，前后两次变换可以首尾咬合，既保留 FFT 正确性，也避免了额外的全局数据搬运。

2.4 数据打包通信

MPI 中若要传输一批逻辑上相关的数据，既可以逐项发送，也可以先将数据写入连续缓冲区后统一发送。实验实现了两种模式：

1. `direct`：逐个复数调用 `MPI_Sendrecv`，作为未优化基线；
2. `packed`：使用 `MPI_Pack` 将本地块装入连续缓冲区，一次发送，再由接收端使用 `MPI_Unpack` 还原。

两种模式的数值结果相同，但通信粒度截然不同。前者会产生大量小消息，消息启动开销占比很高；后者把多次碎片化通信收束为一次块传输，更接近高带宽网络所偏好的访问形态。

三 实验内容

3.1 程序实现

实验在 `lab/07/src/fft_mpi.cpp` 中实现 MPI 并行 FFT，并在 `lab/07/CMakeLists.txt` 中使用 CMake 组织构建。程序主要包含以下部分：

1. 参数解析：支持 `--n`、`--iters` 与 `--mode` 三个命令行参数；
2. 数据分发与回收：使用 `MPI_Scatter` 将输入序列切分给各进程，使用 `MPI_Gather` 汇总验证结果；
3. 局部蝶形运算：在无需通信的阶段直接对本地连续数组执行 DIT 或 DIF；
4. 跨进程蝶形运算：根据当前阶段寻找伙伴进程，并在交换数据后完成跨块合并；
5. 正确性验证：先做一次前向与反向变换，计算恢复结果相对原序列的 RMS 误差；
6. 性能测试：使用 `MPI_Wtime` 计时，并借助 `MPI_Reduce` 取各进程中的最长运行时间作为总耗时。

构建与运行方式如下：

```
cmake -S lab/07 -B lab/07/build
cmake --build lab/07/build

mpirun -np 4 lab/07/build/fft_mpi --n 65536 --iters 5 --mode packed
mpirun -np 4 lab/07/build/fft_mpi --n 65536 --iters 5 --mode direct
```

3.2 正确性验证

在 $N = 65536$ 、迭代次数为 5 的测试中，1、2、4 个进程以及两种通信模式均得到相同量级的误差：

```
RMS error = 2.528540e-16
```

该误差接近双精度浮点数舍入误差范围，说明并行划分、跨进程蝶形以及正反变换配合均保持了数值正确性。

3.3 并行性能与通信方式比较

固定问题规模为 $N = 65536$ ，每次测试执行 5 轮正反变换，结果如下：

进程数	通信模式	总时间 / s	单次变换时间 / s	MFLOPS
1	direct	0.047118	0.004712	1112.716
1	packed	0.043114	0.004311	1216.043
2	direct	0.135386	0.013539	387.254
2	packed	0.028493	0.002849	1840.042
4	direct	0.128908	0.012891	406.715
4	packed	0.021938	0.002194	2389.835

表 1 不同进程数与通信模式下的 FFT 性能

从结果可见，`packed` 模式在 2 个与 4 个进程时均明显优于 `direct` 模式。`direct` 在跨进程阶段为每个复数单独发起通信，随着进程数增加，小消息启动开销迅速吞没了并行收益；`packed` 则将一个本地块打包后一次发送，使通信代价更接近“少量大消息”的模式，因此能够随进程数增加继续获得加速。

若以单进程 `packed` 版本为基准，则 2 进程与 4 进程的加速比分别约为 1.51 与 1.97。其结果低于理想线性加速，原因在于 FFT 的后若干阶段不可避免地需要跨进程交换数据，进程数增加后本地计算量下降，而通信与同步占比随之上升。并行程序的边界并不是算力本身，而是数据被迫穿过进程边界的频率。

3.4 内存分析方法

实验要求使用 Valgrind Massif 采集内存消耗，并在命令中加入 `--stacks=yes` 统计运行栈。可使用如下方式执行：

```
mpirun -np 4 valgrind --tool=massif --stacks=yes \  
  --massif-out-file=massif.out.%p \  
  lab/07/build/fft_mpi --n 65536 --iters 5 --mode packed  
  
ms_print massif.out.<pid>
```

程序中每个进程主要持有本地数据块 `local`、交换缓冲区 `peer`，以及打包模式下的发送与接收缓冲区。若全局规模为 N 、进程数为 P ，则单进程核心数据量约随 $\frac{N}{P}$ 线性变化；`packed` 模式会额外增加两块与本地数据规模同阶的通信缓冲区。因此在 Massif 曲线上，初始化后内存会迅速上升到稳定平台，跨进程阶段进行打包时会出现额外峰值，但总内存仍由 $O(\frac{N}{P})$ 级别的本地存储主导。

四 实验总结

本实验完成了从串行 FFT 到 MPI 并行 FFT 的改造，并通过 `direct` 与 `packed` 两种通信方式展示了一个很有代表性的并行事实：算法能否并行只是第一步，数据如何移动往

往决定它最终跑得多快。前向 DIF 与反向 DIT 的组合减少了额外重排，连续块划分让局部阶段保持良好的缓存访问，而 `MPI_Pack` / `MPI_Unpack` 则将碎片化的小消息压缩为更适合网络传输的大块通信。

从测试结果看，打包通信版本在多进程下明显优于逐项发送版本，但其加速比仍低于理想值，说明 FFT 的跨阶段依赖会逐渐把程序从“计算受限”推向“通信受限”。后续若继续优化，可进一步考虑预先复用通信缓冲区、使用非阻塞通信重叠计算与传输，或采用更贴近硬件拓扑的数据布局。这个实验真正留下的经验，是并行程序不是把循环切开后就结束了；它更像是在计算与通信之间设计一条尽量平顺的河道。